

Segurança do Hub MaisTODOS

Resposta ponto a ponto ao retorno da liderança e briefing técnico do sistema, para instruir a análise do time de segurança.

Data 02/09/2026 Aplicação mais-todos-hub.lovable.app Commit f0eed254

Estado operação em dados fictícios

2 PONTOS FECHADOS	1 DECISÃO DO RESPONSÁVEL	1 VERIFICADO, JÁ CONFORME
4 ACHADOS EM ABERTO		

Resposta ao retorno da liderança

Os quatro pontos levantados na revisão, na ordem em que foram colocados, com o que foi feito em cada um.

P1

Cabeçalhos de iframe e clickjacking

DECISÃO DO RESPONSÁVEL

O QUE FOI APONTADO

A aplicação publicada não manda X-Frame-Options nem CSP, então qualquer site consegue embutir o painel num iframe. A correção seria uma linha de frame-ancestors.

O diagnóstico do risco está correto, a correção não é de uma linha. A aplicação passou a emitir quatro cabeçalhos de segurança em toda resposta, inclusive nas de erro, aplicados em `src/server.ts`. Depois de publicar, medi o que chega ao navegador.

CABEÇALHO	SAI DO SERVIDOR	CHEGA AO NAVEGADOR
X-Content-Type-Options	sim	sim
Referrer-Policy	sim	sim
Content-Security-Policy	sim	não
X-Frame-Options	sim	não

Os quatro saem do mesmo ponto do código, e só os dois que impedem o enquadramento em iframe desaparecem. Medido na URL publicada e no preview, em rota de aplicação, em rota inexistente e em arquivo estático, sempre com o mesmo resultado. A remoção é da camada de borda da hospedagem, que precisa embutir a aplicação em iframe no próprio editor. Não existe correção por código enquanto o domínio for `lovable.app`.

SITUAÇÃO

O clickjacking continua possível hoje. Decisão do responsável: manter os cabeçalhos no código, sem trava adicional no cliente, e fechar o ponto na migração para domínio próprio atrás da CDN da empresa, onde eles passam a valer sem nenhuma alteração. O risco está atenuado porque o atendimento opera em dados fictícios, sem integração com as APIs reais. Precisa estar fechado antes da integração.

P2 Políticas ausentes do repositório

FECHADO

O QUE FOI APONTADO

As políticas de profiles, user_roles e kb_documents não vieram no export, as migrations começam em 06/08 e essas são anteriores. Sem elas ninguém aprova nem reprova o painel lendo o repositório.

Confirmado, e a causa é estrutural: o Hub nasceu de um remix de outro projeto, então o schema veio pronto e as migrations versionadas só registram o que foi criado a partir de 06/08. Ler o repositório nunca vai mostrar essas políticas.

Extraí o estado real do banco e produzi o documento **Hub MaisTODOS, dump de políticas RLS e privilégios**, que acompanha o repositório na entrega. Ele traz as políticas das dez tabelas e do storage, o estado de RLS por tabela, os privilégios concedidos a cada papel, as funções próprias com o respectivo `search_path`, e as limitações assumidas. O resumo técnico está nas seções 4 a 6 deste briefing.

P3 Permissão ampla da extensão

FECHADO

O QUE FOI APONTADO

O `all_urls` é amplo demais, mas o código não coleta nada: injeta um botão e lê só o texto selecionado no clique. Vale reduzir para `activeTab`, sem urgência.

A leitura do código está certa, e a correção foi aplicada junto com P1, no mesmo build. O bloco `content_scripts` saiu do manifesto e os arquivos `content.js` e `content.css` foram removidos. A extensão mantém `activeTab`, concedida apenas

no clique do usuário, e o pacote distribuível em `public/maisia-copiloto.zip` foi refeito, senão o download continuaria entregando a versão antiga.

Os dois caminhos de uso seguem funcionando sem permissão ampla: o clique no ícone, que abre o painel lateral, e o menu de contexto sobre um texto selecionado. O que se perdeu foi o botão flutuante injetado nas páginas.

CORREÇÃO DE PREMISSA

A extensão não é prova de que o Hub pode ser embutido. O painel lateral dela carrega `mais-tudo-mais-sabio.lovable.app/maisia`, que é a aplicação de origem do remix, não o Hub. Ela prova que uma aplicação hospedada na plataforma pode ser embutida, o que sustenta o risco de P1, mas hoje ela não embute o Hub.

P4 **Confirmação de e-mail**

VERIFICADO, JÁ CONFORME

O QUE FOI APONTADO

A trava de domínio garante o domínio, não que a pessoa tenha a caixa. Confere se a confirmação de e-mail está ligada.

Está ligada, verificado no estado das contas. Os dois cadastros por senha receberam o e-mail de confirmação no ato do cadastro. Um deles só teve a confirmação registrada 45 segundos depois, e entrou apenas em seguida. O cadastro de 31/08 segue com confirmação pendente e nunca logou. As outras dezesseis contas entraram por Google, onde a confirmação vem do próprio provedor.

O que é o Hub

Ferramenta interna de uso corporativo, acessível apenas a colaboradores autenticados do domínio `maistodos.com.br`. Reúne duas naturezas de conteúdo que seguem regras de acesso diferentes, e essa separação é a espinha do modelo de segurança.

- **Conteúdo, aberto a todo autenticado.** Universidade corporativa, base de conhecimento, processos, fluxogramas, scripts, FAQ e o guia de operação de todos os setores em modo consulta. É decisão de produto: qualquer pessoa precisa poder estudar qualquer setor.
- **Operação, restrita por papel.** Módulos de atendimento (filiados, cashback, estornos, carteirinhas), painel de nível 2, auditoria, gestão de acessos e treino da base de IA.

Os módulos de operação hoje trabalham com **dados fictícios**. Não há integração com as APIs reais de filiado, carteira ou cashback. A integração está congelada até a aprovação do projeto, e é o marco que muda o perfil de risco de vários itens deste documento.

ITEM	SITUAÇÃO
Aplicação	TanStack Start com SSR, React e Vite, hospedada na plataforma Lovable
Banco	PostgreSQL gerenciado (Supabase), com RLS em todas as tabelas
Autenticação	Supabase Auth, login por Google e por senha
Busca semântica	pgvector, embeddings e chat por gateway da plataforma
Contas	18 contas, 10 com papel de administrador
Repositório	GitHub, sem resposta anônima na API, comportamento de repositório privado

Arquitetura e fronteira de confiança

Este é o ponto que a análise precisa entender antes de tudo, e a liderança já o identificou corretamente: o navegador carrega a chave pública e o JWT do usuário, então qualquer pessoa autenticada pode conversar com a API do banco direto, sem passar pela aplicação. Toda checagem feita apenas na interface é cosmética.

Navegador	Renderiza a interface e esconde o que a pessoa não pode operar. Guarda de rota impede acesso por URL direta. Nada disso é controle de segurança , é experiência de uso.
Funções de servidor	Autenticam, conferem o papel com <code>has_role</code> e só então usam a chave privilegiada. É por aqui que passam concessão de papéis, liberação de módulos, sincronização da base e chamadas às integrações externas.
Banco, RLS	Fronteira real. Vale igual para quem usa a aplicação e para quem chama a API direto com o JWT. Se um controle não existe aqui, ele não existe.
Integrações	Notion e provedores de IA são acionados somente pelo servidor, com token lido do cofre de segredos. O navegador não fala com serviço externo.

A pergunta que a análise deve fazer em cada tabela é sempre a mesma: com um JWT de colaborador comum e a chave pública, o que essa política deixa ler e escrever. A seção seguinte responde isso.

Identidade e acesso

Quem consegue entrar

A restrição de domínio é um gatilho `BEFORE INSERT` e `BEFORE UPDATE` em `auth.users`, não validação de formulário. Isso importa porque cobre também o login por Google, que cria a linha diretamente. Cadastro fora de `maistodos.com.br` é recusado no banco. Confirmação de e-mail ativa, conforme P4.

Existe uma conta de domínio externo criada em 26/08, anterior à trava de 31/08. A permanência dela foi decisão explícita do responsável.

Escalada de privilégio

A tabela de papéis tem **apenas políticas de SELECT**. Não há INSERT, UPDATE nem DELETE para o cliente, então ninguém escreve o próprio papel, nem chamando a API do banco direto. Toda concessão passa por função de servidor que valida papel de administrador, com trava contra remover o próprio acesso e contra remover o último administrador.

A função de auto promoção que existia, no padrão primeiro usuário vira administrador, foi removida em 31/08. Não há nenhum caminho de auto concessão remanescente.

Liberação de módulo da operação vive em tabela própria, também só com SELECT para o cliente, escrita apenas pelo servidor.

Dados, RLS e privilégios

RLS habilitado nas dez tabelas do schema público, sem exceção. Todas as políticas se aplicam ao papel autenticado. **Nenhuma política e nenhum privilégio concede acesso ao papel anônimo**, em nenhuma tabela.

TABELA	LEITURA	ESCRITA PELO CLIENTE
profiles	só o próprio perfil	só o próprio perfil
user_roles	os próprios papéis, administrador vê todos	nenhuma
user_area_accesses	as próprias liberações, administrador vê todas	nenhuma
kb_documents	documento ativo, arquivado só administrador	só administrador
kb_chunks	trecho de documento ativo, pelo filtro do pai	só administrador
kb_categories	qualquer autenticado, são rótulos sem conteúdo	só administrador
kb_document_versions	só administrador	só administrador
atendimento_auditoria	as próprias ações, tudo para gestor e administrador	apenas inserir a própria ação
ai_queries	as próprias consultas, tudo para administrador	apenas inserir a própria consulta
ai_usage	só administrador	nenhuma, gravação só no servidor

Onde o privilégio de tabela parece largo, é a política que fecha. Em `kb_documents` o papel autenticado tem privilégio de escrita, mas a política exige administrador, então o colaborador comum recebe zero linha afetada. Não existe `TRUNCATE` concedido em lugar nenhum, o que importa porque `TRUNCATE` não passa por RLS. O `GRANT` amplo que existia foi revogado em 01/09.

Funções que contornam a RLS

As seis funções escritas para o projeto são `SECURITY DEFINER` com `search_path` fixo em `public`. Duas delas são de busca, vetorial e textual, e por serem definir passam por cima das políticas. Ambas repetem internamente o filtro `status = 'ativo'` e não aceitam parâmetro que permita alcançar documento arquivado.

A trilha de auditoria só admite inserir e ler. Não há `UPDATE` nem `DELETE` para ninguém, então registro não é editado nem apagado pela via do cliente.

Dados pessoais

O sistema lida com CPF, e-mail e telefone de filiados nos módulos de atendimento, hoje fictícios, e com dados de colaboradores no conteúdo interno.

- **Mascaramento na gravação.** O payload da trilha de auditoria passa por uma função única de sanitização antes de gravar e antes de espelhar. CPF e e-mail não vão completos para a trilha.
- **Retenção.** Existe rotina de expurgo de doze meses para os registros de consulta e de consumo de IA. A trilha de auditoria está deliberadamente fora do expurgo, com o prazo de guarda pendente de definição jurídica.
- **Texto livre.** Pergunta de chat e campo de observação são a maior fonte de vazamento em sistemas assim, porque o conteúdo é digitado pela pessoa. A pergunta é gravada e enviada ao provedor de IA. Vale a análise olhar esse ponto com atenção.
- **Dados de colaborador no pacote da aplicação.** O guia de operação inclui nome, cargo, telefone e e-mail corporativo de pessoas reais em arquivo de dados do frontend, o que significa que vai no pacote entregue ao navegador e é legível por qualquer colaborador autenticado, independente do que a interface mostra. É coerente com a decisão de produto de conteúdo aberto, e é o motivo pelo qual o repositório precisa ser privado.

Segredos e superfície de rede

A chave privilegiada do banco é usada apenas em funções de servidor, dentro de fluxo que já conferiu o papel. Os tokens de integração, incluindo os do Notion, vivem no cofre de segredos do ambiente e são lidos com `process.env` no servidor. O módulo de integração redige o token nas mensagens de erro, para não vazarem em log.

Nenhum token de API foi encontrado em código. O arquivo `.env.example` documenta as variáveis sem valores e diz explicitamente para não colocar a chave de serviço ali.

PONTO DE ATENÇÃO

O arquivo `.env` existe no repositório e não consta do `.gitignore`. Pelo modelo documentado, ele carrega apenas identificadores do projeto e a chave publicável, que é protegida por RLS e já vai para o navegador de qualquer forma, ou seja, não é exposição de segredo forte. Ainda assim é prática que o time de segurança vai apontar, e a correção é uma linha no `.gitignore` mais a remoção do arquivo do versionamento.

SITUAÇÃO

Achados em aberto

O que segue aberto depois das correções de 31/08, 01/09 e 02/09. Nenhum deles é defeito de implementação recente: são limites conhecidos, três dos quais dependem de decisão e não de código.

SEVERIDADE	ACHADO	EFEITO	TRATAMENTO
ALTO	Cabeçalho anti iframe removido pela hospedagem	Clickjacking sobre ações de atendimento com o atendente logado	Migração para domínio próprio, decisão tomada
MÉDIO	Auditoria gravada pelo navegador	Quem abrir o console pode bloquear o registro e agir sem rastro. Ninguém falsifica registro alheio nem apaga registro existente	Mover a gravação para a função de servidor que executa a ação, junto com a integração real
MÉDIO	Dez administradores em dezoito contas	O modelo de papéis rende pouco quando a maioria vê tudo	Decisão de gestão, não é defeito técnico
BAIXO	<code>.env</code> versionado, fora do <code>.gitignore</code>	Sem segredo forte exposto, é má prática de versionamento	Ignorar o arquivo e removê-lo do versionamento
BAIXO	Uma conta fora do domínio corporativo	Anterior à trava, que hoje impede casos novos	Permanência decidida pelo responsável
BAIXO	Extensão aponta para a aplicação de origem do remix	Confusão de escopo na revisão, sem efeito de segurança	Repontar quando a extensão passar a servir o Hub

Corrigido antes desta entrega

Para contexto da análise, estes pontos foram encontrados e fechados nos dias anteriores: cadastro aberto a qualquer e-mail, agora travado no banco; função de auto promoção a administrador, removida; políticas permissivas em tabelas de trechos e categorias, substituídas por filtro herdado do documento; dado pessoal completo no payload da auditoria e nos registros de IA, sanitizado; e privilégio amplo que concedia TRUNCATE aos papéis do cliente, revogado.

PRÓXIMO MARCO

O que muda na integração real

Hoje o atendimento é uma casca sobre dados fictícios. Quando as APIs de filiado, carteira e cashback entrarem, três coisas mudam de peso e precisam estar resolvidas antes:

- O clickjacking deixa de ser teórico, porque a ação induzida passa a produzir efeito em sistema produtivo. É o item que amarra a migração para domínio próprio.
- A gravação da auditoria precisa acontecer no servidor, junto da execução da ação, senão o rastro fica opcional justamente quando passa a importar.
- As credenciais das APIs externas passam a existir no ambiente. Nenhuma delas pode chegar ao navegador, e a chamada precisa sair da camada de servidor.

Como verificar

Tudo que este briefing afirma sobre o banco foi lido do banco, não do repositório. A consulta que reproduz o dump de políticas está no documento anexo, junto do estado de RLS por tabela, dos privilégios por papel e da definição das funções próprias.

A medição dos cabeçalhos é reproduzível com uma requisição simples à URL publicada, comparando os quatro cabeçalhos emitidos em `src/server.ts` com o que chega na resposta.

ENTREGAR JUNTO

Repositório da aplicação, privado. Documento **Hub MaisTODOS, dump de políticas RLS e privilégios**, de 02/09/2026. Este briefing.

Documento interno MaisTODOS, preparado para a revisão de segurança do Hub. Os números de contas e papéis refletem o estado do banco em 02/09/2026 e mudam conforme a gestão de acessos.